

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №10

Индексаторы

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4321

подпись, дата

Г.В.Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 Цель работы.....	3
2 Задание.....	3
3 Описание работы программы	4
4 Результаты работы программы.....	10
5 Вывод	11
Приложение А. Листинг Программы.....	12

1 Цель работы

Цель данной работы заключается в изучении и практическом применении индексаторов в языке программирования C#. Индексаторы позволяют объектам вести себя как массивы, предоставляя удобный способ доступа к элементам коллекции через квадратные скобки.

2 Задание

Реализовать индексы в классе `WheelCollection`, который будет хранить массив объектов `Wheel`.

3 Описание работы программы

В процессе разработки приложения для управления автопарком мы столкнулись с необходимостью упрощения доступа к компонентам автомобиля, особенно к колесам. Для решения этой задачи мы решили реализовать индексаторы, которые позволяют обращаться к элементам коллекции, как если бы это были массивы. Это значительно улучшает читаемость и удобство работы с кодом. Мы создали класс `WheelCollection`, который хранит массив объектов `Wheel` и предоставляет индексатор для доступа к каждому колесу. На рисунке 1 изображен класс `WheelCollection`.

```
using System;

Ссылка 1
public class WheelCollection
{
    private Wheel[] wheels;

    Ссылка 0
    public WheelCollection(int size)
    {
        wheels = new Wheel[size];
        for (int i = 0; i < size; i++)
        {
            wheels[i] = new Wheel(); // Инициализация каждого колеса
        }
    }

    Ссылка 0
    public Wheel this[int index]
    {
        get
        {
            if (index < 0 || index >= wheels.Length)
                throw new IndexOutOfRangeException("Индекс вне диапазона.");
            return wheels[index];
        }
        set
        {
            if (index < 0 || index >= wheels.Length)
                throw new IndexOutOfRangeException("Индекс вне диапазона.");
            wheels[index] = value;
        }
    }

    Ссылка 0
    public int Count => wheels.Length; // Свойство для получения количества колес
}
```

Рисунок 1 – класс `WheelCollection`

Индексатор в классе `WheelCollection` был реализован с использованием стандартного синтаксиса C#. Он позволяет пользователю получать и

устанавливать состояние каждого колеса, используя квадратные скобки. Это упрощает взаимодействие с классом, так как теперь не нужно использовать методы для доступа к отдельным колесам. Вместо этого мы можем просто обращаться к ним по индексу, что делает код более лаконичным и понятным. На рисунке 2 изображен класс Car.

```
public class Car
{
    public string Name { get; set; }
    public int HorsePower { get; set; }
    public int FuelConsumption { get; set; }
    public int FuelTankCapacity { get; set; }
    public int Fuel { get; set; }
    public int Mileage { get; set; }
    public Engine Engine { get; set; }
    public WheelCollection Wheels { get; set; }

    public Car(string name, int horsePower, int fuelConsumption,
    {
        Name = name;
        HorsePower = horsePower;
        FuelConsumption = fuelConsumption;
        FuelTankCapacity = fuelTankCapacity;
        Fuel = 0; // Начальное количество топлива
        Mileage = 0; // Начальный пробег
        Engine = new Engine();
        Wheels = new WheelCollection(4); // Предполагаем, что у
    }
```

Рисунок 2 – изменения класса Car

После создания класса WheelCollection мы внесли изменения в класс Car, чтобы интегрировать новый класс в его структуру. Вместо использования простого массива колес мы теперь используем экземпляр WheelCollection. Это изменение позволило нам использовать индексаторы для управления состоянием колес автомобиля. Например, мы можем легко установить состояние первого колеса в "сломано" или получить информацию о состоянии второго колеса, просто указав его индекс. На рисунке 3

изображено тестирование.

```
Car myCar = new Car("Машина 1", 150, 10, 50);  
myCar.Wheels[0].ChangeCondition("Сломано"); // Изменение состояния  
string firstWheelCondition = myCar.Wheels[0].Condition; //
```

Рисунок 3 – проверка работа индексатора

Реализация индексаторов не только улучшила структуру кода, но и сделала его более интуитивным для разработчиков, работающих с приложением. Теперь, когда мы добавляем новые функции или изменяем существующие, мы можем легко управлять компонентами автомобиля, не беспокоясь о сложных методах доступа. Это значительно ускоряет процесс разработки и упрощает поддержку кода в будущем. В результате, использование индексаторов в нашем приложении стало важным шагом к созданию более эффективного и удобного в использовании программного обеспечения. На рисунке 4 изображен результат тестирования.

Сломано

Рисунок 4 – результат проверки работы индексатора

На рисунке 5 изображена диаграмма классов программы.

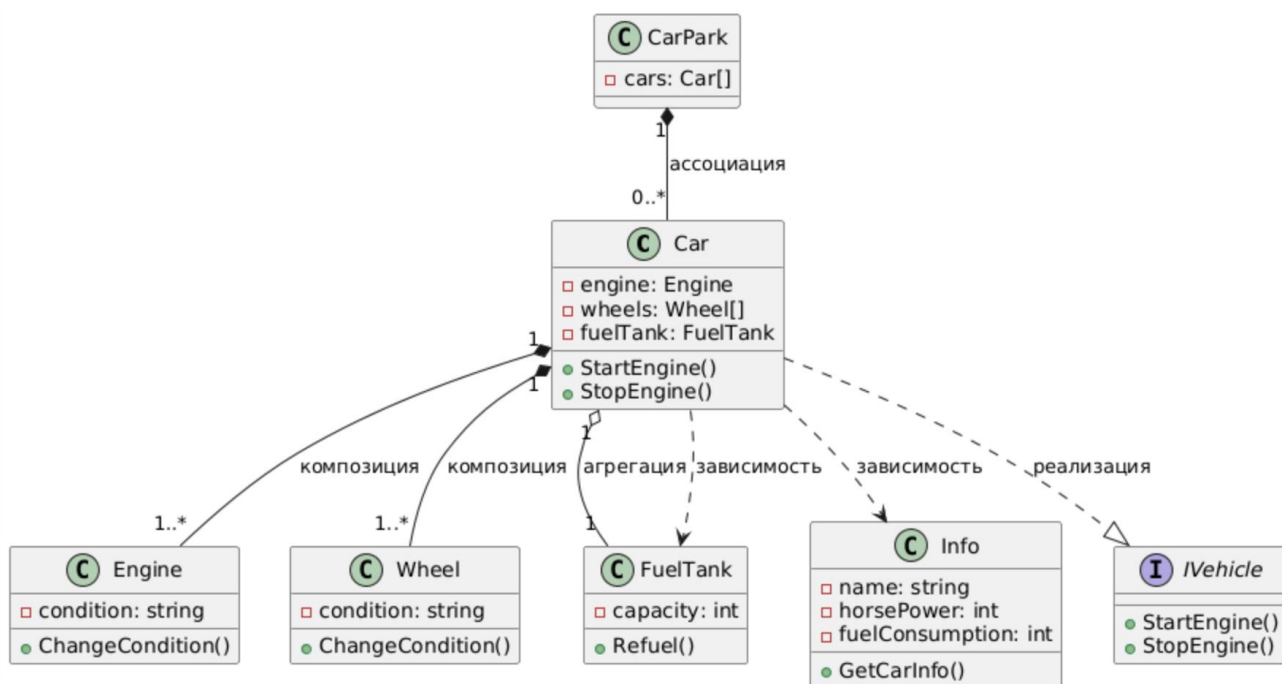


Рисунок 5 – Диаграмма классов

На рисунке 6 представлена структура оформления базового окна Windows Forms.

The screenshot shows a Windows Forms application window titled "Car Management". The window has a standard Windows title bar with minimize, maximize, and close buttons. The main area contains several controls:

- On the left, there are four text labels with corresponding input fields: "Введите имя авто:", "Введите мощность авто:", "Введите расход авто:", and "Введите бак авто:". Below these are two buttons: "Создать авто" and "Удалить авто". At the bottom left, there is a checkbox labeled "Подробно" and a button labeled "Показать".
- In the center, there is a large, empty rectangular box.
- On the right, there are four buttons stacked vertically: "Заправка", "Передвижение", "Запустить", and "Выключить". Below these is a button labeled "Замениь".
- At the top right, there is a checkbox labeled "Высокая скорость".

Рисунок 6 – Оформление окна Windows Forms

4 Результаты работы программы с примерами

На рисунке 7 представлен результат работы программы показа работы индексаторов компонента.

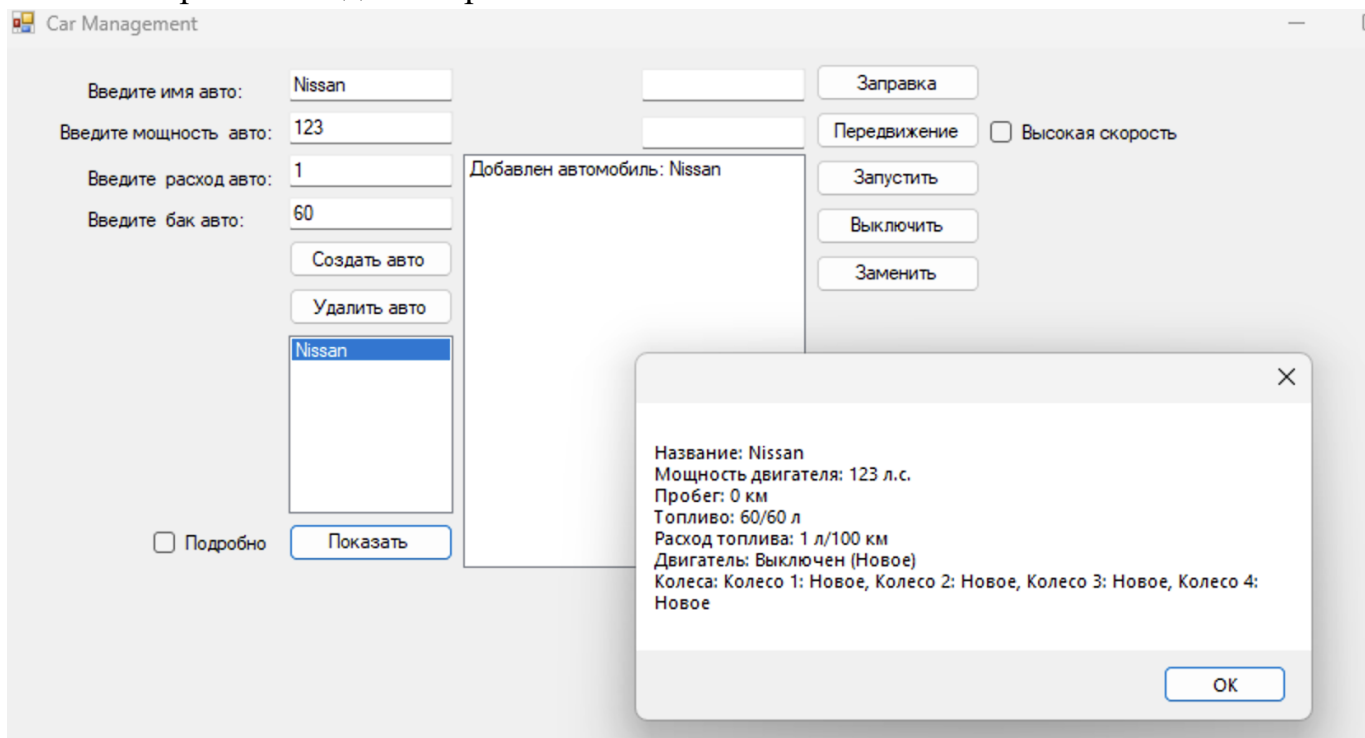


Рисунок 7 – Результат работы программы

5 Вывод

Реализация индексаторов позволила нам интегрировать их в класс Car, заменив традиционный массив колес на экземпляр WheelCollection. Это изменение упростило взаимодействие с компонентами автомобиля, так как теперь мы можем легко изменять и получать состояние каждого колеса, используя синтаксис, аналогичный массивам. Например, мы можем просто указать индекс колеса, чтобы изменить его состояние или получить информацию о нем, что делает код более лаконичным и интуитивно понятным.

Кроме того, использование индексаторов способствовало улучшению структуры кода и его поддерживаемости. Мы смогли избежать избыточности и упростить логику управления автомобилем, что в свою очередь ускорило процесс разработки и тестирования. Это также позволило нам сосредоточиться на других аспектах приложения, не отвлекаясь на сложные методы доступа к элементам.

В результате работы мы не только углубили свои знания о индексаторах в C#, но и улучшили функциональность нашего приложения. Реализация индексаторов стала важным шагом к созданию более эффективного и удобного в использовании программного обеспечения, что подтверждает их полезность в объектно-ориентированном программировании. Мы уверены, что полученные знания и опыт будут полезны в будущих проектах и помогут в дальнейшей разработке программного обеспечения.

Приложение А. Листинг Программы

Файл «Component»

```
using System;

public abstract class Component
{
    // Абстрактный класс Component, который реализует общие методы и события для всех компонентов автомобиля
    // Свойство Condition хранит текущее состояние компонента, по умолчанию "Новое"
    public string Condition { get; protected set; } = "Новое";

    // Метод ChangeCondition для изменения состояния компонента
    // Проверяет, что состояние может быть только "Новое" или "Сломано"
    // При изменении состояния на "Сломано" вызывает событие поломки компонента
    public void ChangeCondition(string condition)
    {
        if (condition != "Новое" && condition != "Сломано")
            throw new ArgumentException("Неверное состояние компонента");

        Condition = condition;

        // Если состояние компонента "Сломано", генерируем событие поломки
        if (Condition == "Сломано")
        {
            OnComponentBroken();
        }
    }

    // Защитный метод для вызова события поломки компонента
    // Срабатывает, если компонент сломался
    protected virtual void OnComponentBroken()
    {
        ComponentBroken?.Invoke(this, EventArgs.Empty); // Генерация события ComponentBroken
    }

    // Событие поломки компонента, которое оповещает об изменении состояния на "Сломано"
```

Файл «IComponent»

```
using System;

public interface IComponent
{
    // Свойство Condition представляет текущее состояние компонента (например, "Новое", "Сломано")
    string Condition { get; set; }

    // Метод для изменения состояния компонента
    // Принимает строку, которая указывает новое состояние компонента (например, "Сломано" или "Новое")
    void ChangeCondition(string condition);
}
```

Файл «Info»

```
using System.Linq;

namespace CarManagementApp
{
    public sealed class Info : Car
    {

```

```

// Конструктор для класса Info, который вызывает конструктор базового класса
public Info(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
    : base(name, horsepower, fuelConsumption, fuelTankCapacity)
{
}

// Переопределяем метод получения информации о машине
public override string GetCarInfo(bool showDetailed = false)
{
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}
}

```